AIGenUT: Intelligent Unit Test Automation

Automating coverage, enforcing team style, and validating with absolute confidence.

NotebookLM

Testing shouldn’t be a bottleneck

Writing and maintaining manual unit tests results in low coverage, inconsistent styles, and tedious log-reading.

`AIGenUT` acts as your intelligent co-pilot, designed specifically to build high-quality, style-consistent test suites without the friction.

NotebookLM

The AIGenUT Architecture

A targeted, intelligent, and secure three-step workflow.

Local Visual Detection

Identify missing coverage.
Scan files locally.
No upload needed.

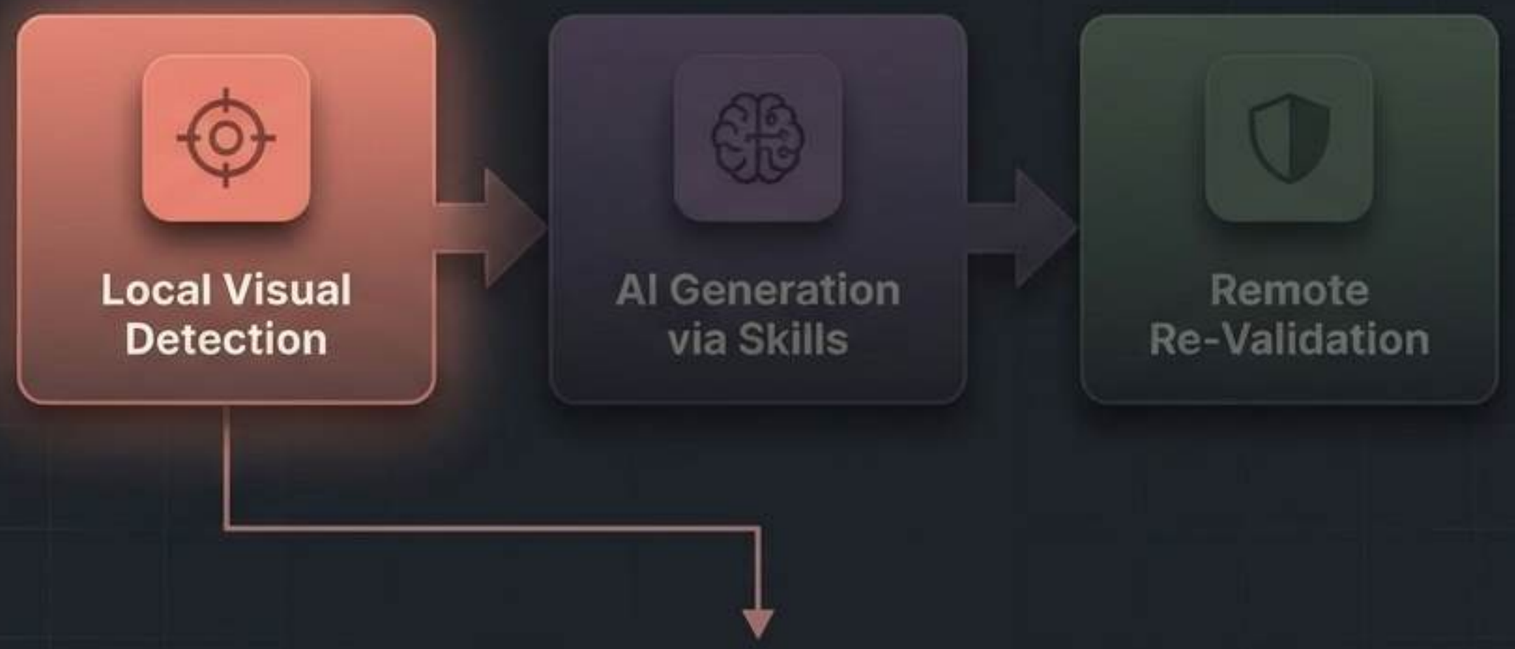
AI Generation via Skills

Synthesize unit tests.
Leverage skill modules.
Privacy focused generation.

Remote Re-Validation

Confirm functionality.
Execute secure checks.
Ensure robust results.

NotebookLM



Step 1: Spot the weak areas without digging through logs.

A purely visual approach to codebase health. Identify exactly what `classes` and `methods` need attention before invoking the AI.

NotebookLM

Local Visual Detection → AI Generation via Skills → Remote Re-Validation

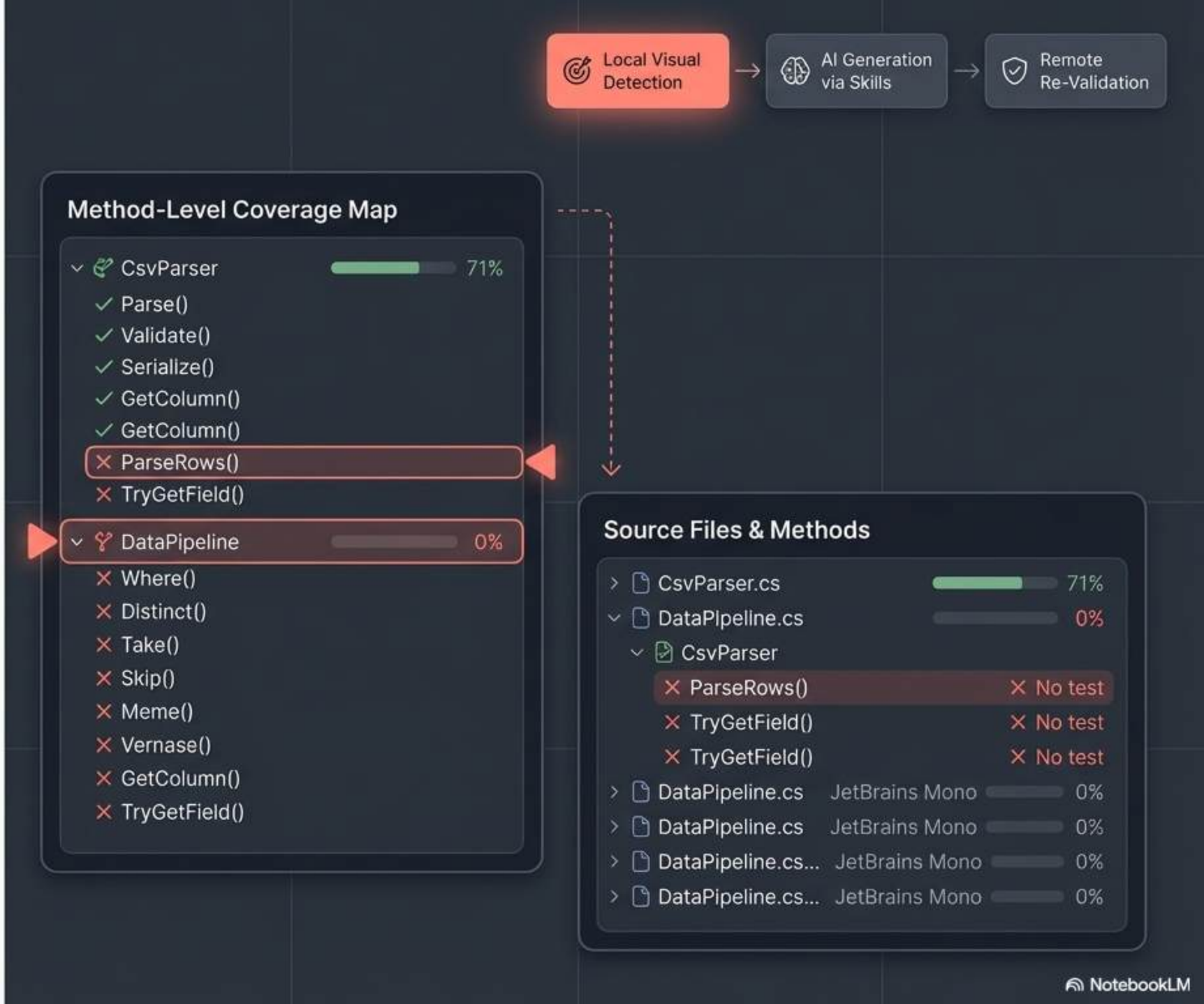
The Macro View: Instant Visibility

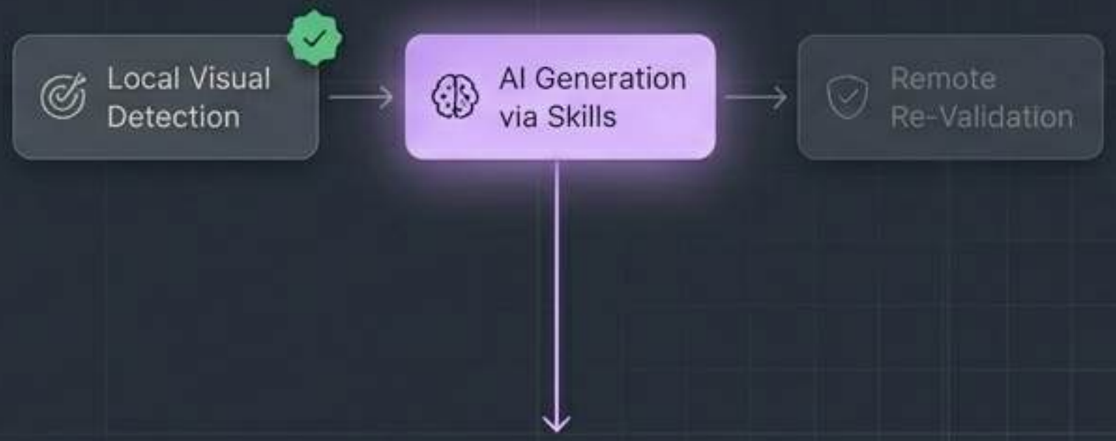
See the health of your codebase at a glance. We eliminate the need to parse complex test logs by surfacing file, method, and line coverage in a clean visual dashboard.

NotebookLM

The Micro View: Pinpoint the Gaps

Green means tested. Red means missing. AIGenUT maps your codebase at the method level, telling the AI exactly where to focus its generation efforts.

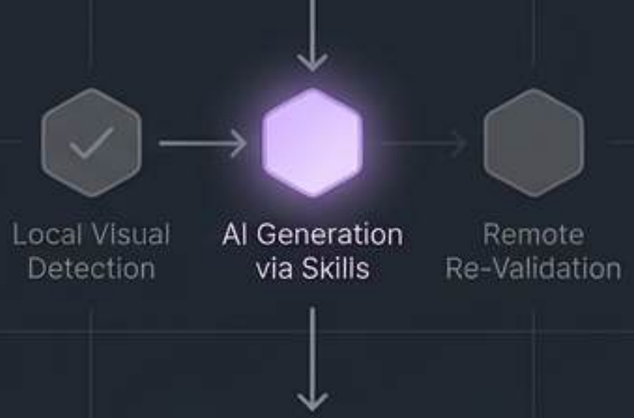




Step 2: AI generation that understands your team's DNA.

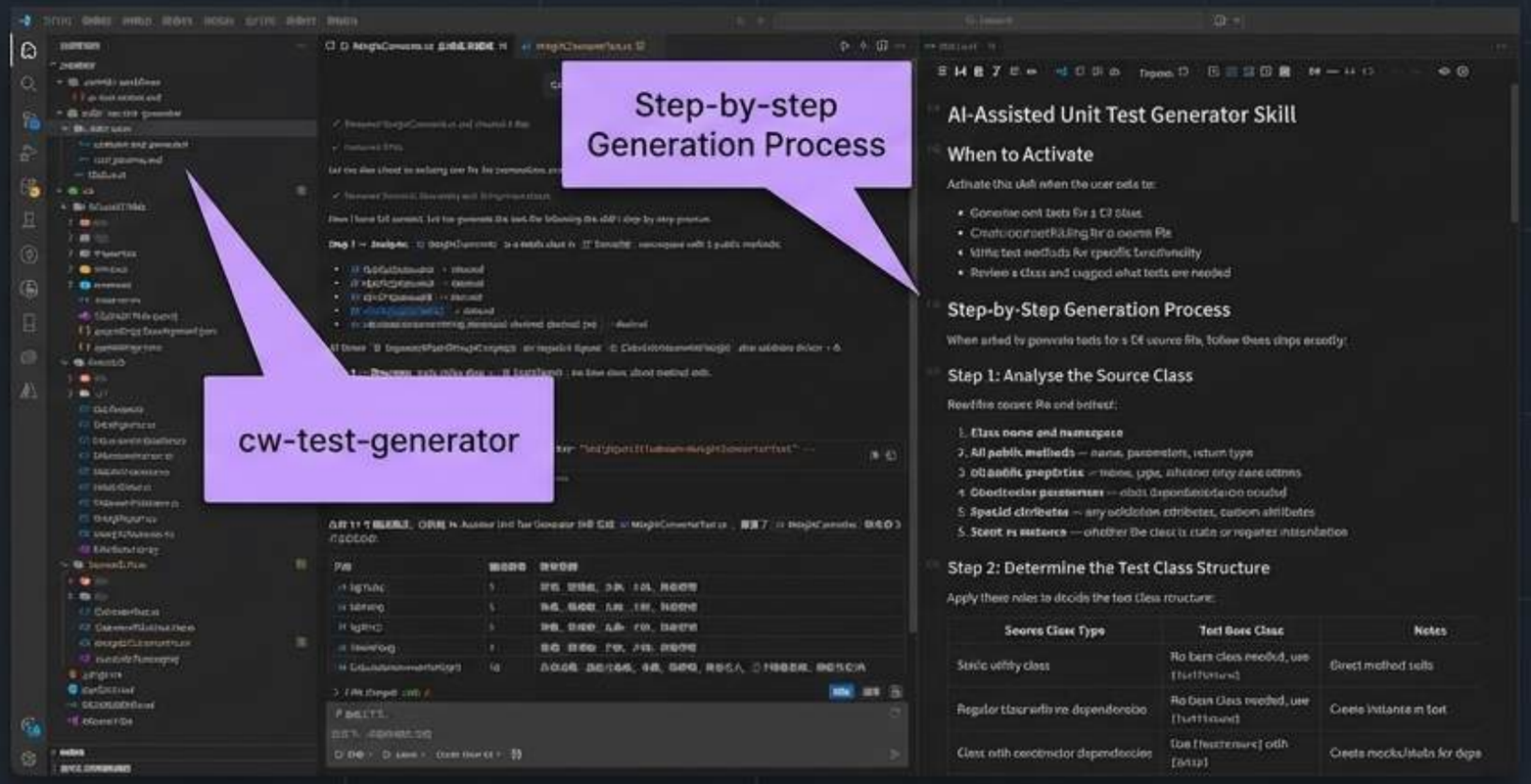
Moving beyond generic AI to produce targeted, style-matched code.

NotebookLM



The Engine: What is a Skill?

A Skill is a curated set of rules and patterns. It teaches the AI to write tests in your specific style (e.g., CargoWise) rather than generic boilerplates—enforcing strict naming conventions and structural integrity.



NotebookLM

The Skill Advantage

Results in a 30% to 50% improvement in both coverage and style matching.

AIGenUT with Skills

```
Yes, 2026-03-08 | Features (Doc and one other)
using NUnit.Framework;

Yes, 2026-03-08 | Features (Doc and one other)
namespace Beerlib.Tests;

/// <summary>
/// Unit tests for WeightConverter - covers all public conversion methods
/// including happy path, edge cases, error cases, and round-trip conversions.
/// </summary>
Yes, 2026-03-08 | Features (Doc and one other)
[TestFixture]
0 references
public class WeightConverterTest
{
    // ------------------------------------------------------------
    // Kg to lbs
    // ------------------------------------------------------------

    [Test]
    0 references
    public void TestKgToLbs_WhenZero_ThenReturnsZero()
    {
        var result = WeightConverter.KgToLbs(0m);
        Assert.That(result, Is.EqualTo(0m));
    }

    [Test]
    0 references
    public void TestKgToLbs_WhenOneKg_ThenReturnsCorrectLbs()
    {
        var result = WeightConverter.KgToLbs(1m);
        Assert.That(result, Is.EqualTo(2.2046m));
```

- Targets identified gaps, matches CargoWise style perfectly.

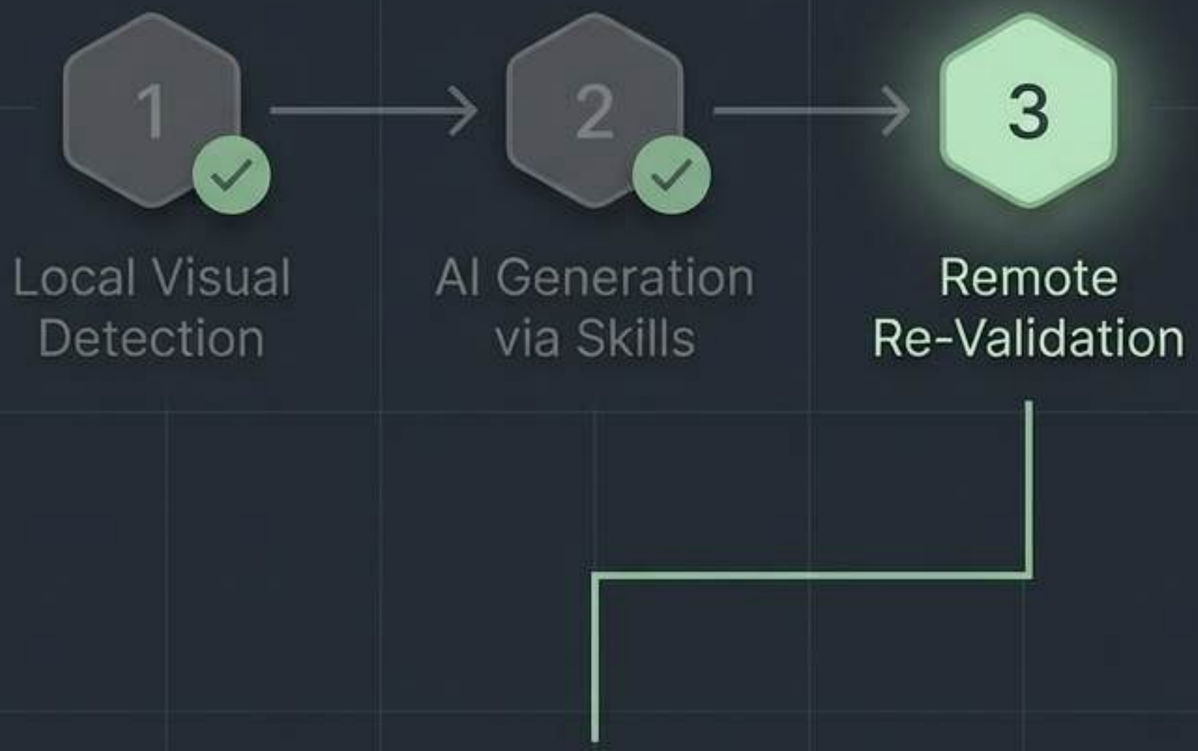
Up to 50% Improvement

Standard AI

```
using Beerlib;
using NUnit.Framework;

namespace Beerlib.Tests;

[TestFixture]
0 references
public class WeightConverterTests
{
    #region Kg to lbs

    [Test]
    0 references
    public void KgToLbs_Zero_ReturnsZero()
    {
        Assert.That(WeightConverter.KgToLbs(0m), Is.EqualTo(0m));
    }

    [Test]
    0 references
    public void KgToLbs_PositiveValue_ReturnsCorrectResult()
    {
        // 1 kg = 2.20462 lbs (rounded to 4 decimals)
        Assert.That(WeightConverter.KgToLbs(5m), Is.EqualTo(11.0230m));
    }

    [Test]
    0 references
    public void KgToLbs_LargeValue_ReturnsCorrectResult()
    {
        // 100 kg = 220.462 lbs - rounded to 220.4620
        Assert.That(WeightConverter.KgToLbs(100m), Is.EqualTo(220.4620m));
    }
```

- Generic tests, low coverage, ignores team conventions.

NotebookLM



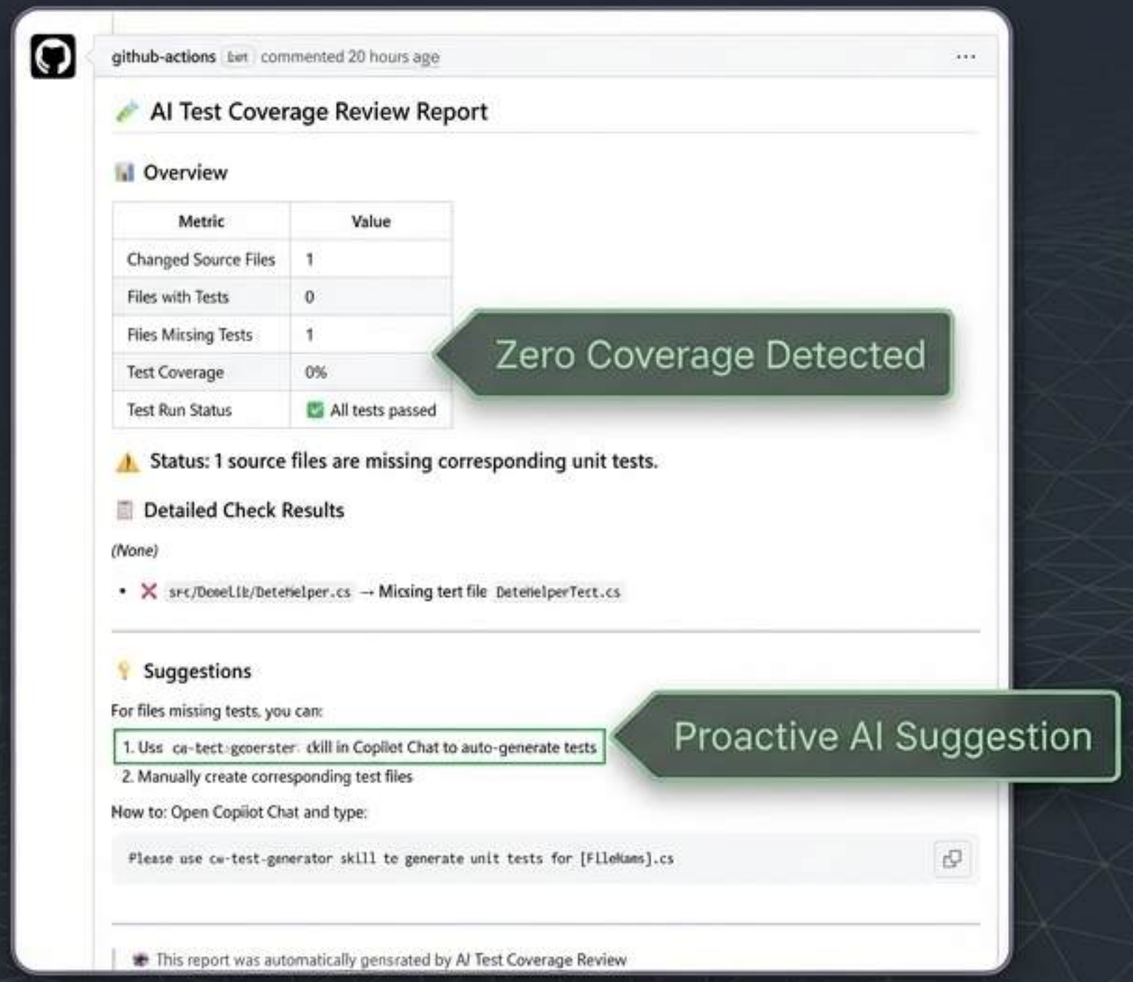
Step 3: The Safety Net. Because we don't trust blindly.

Local generation is fast, but `remote validation` guarantees absolute confidence before you merge.

NotebookLM

Remote Re-Validation via GitHub Actions

Push your code to GitHub, and our pipeline recalculates coverage in a clean, remote environment. It validates local numbers, catches missed files, and proactively suggests AI fixes directly in the PR.



NotebookLM

The Precision Pipeline Complete

Local Visual Detection

Identify the gaps locally.

AI Generation via Skills

Generate code intelligently with Skills.

Remote Re-Validation

Validate remotely with total confidence.

Three steps to flawless code coverage.

NotebookLM

Transform Your Testing Workflow

Ready to experience intelligent test automation? Explore the codebase, review the Skills integration, and run the pipeline yourself.

```
>_ https://github.com/Eden-dot-Ye/AIGenUT
```

NotebookLM